# Báo cáo LAB-02 – Viết Device Driver & Quản lý File System

Oct 6, 2026 · @Nguyễn Ngọc Quang

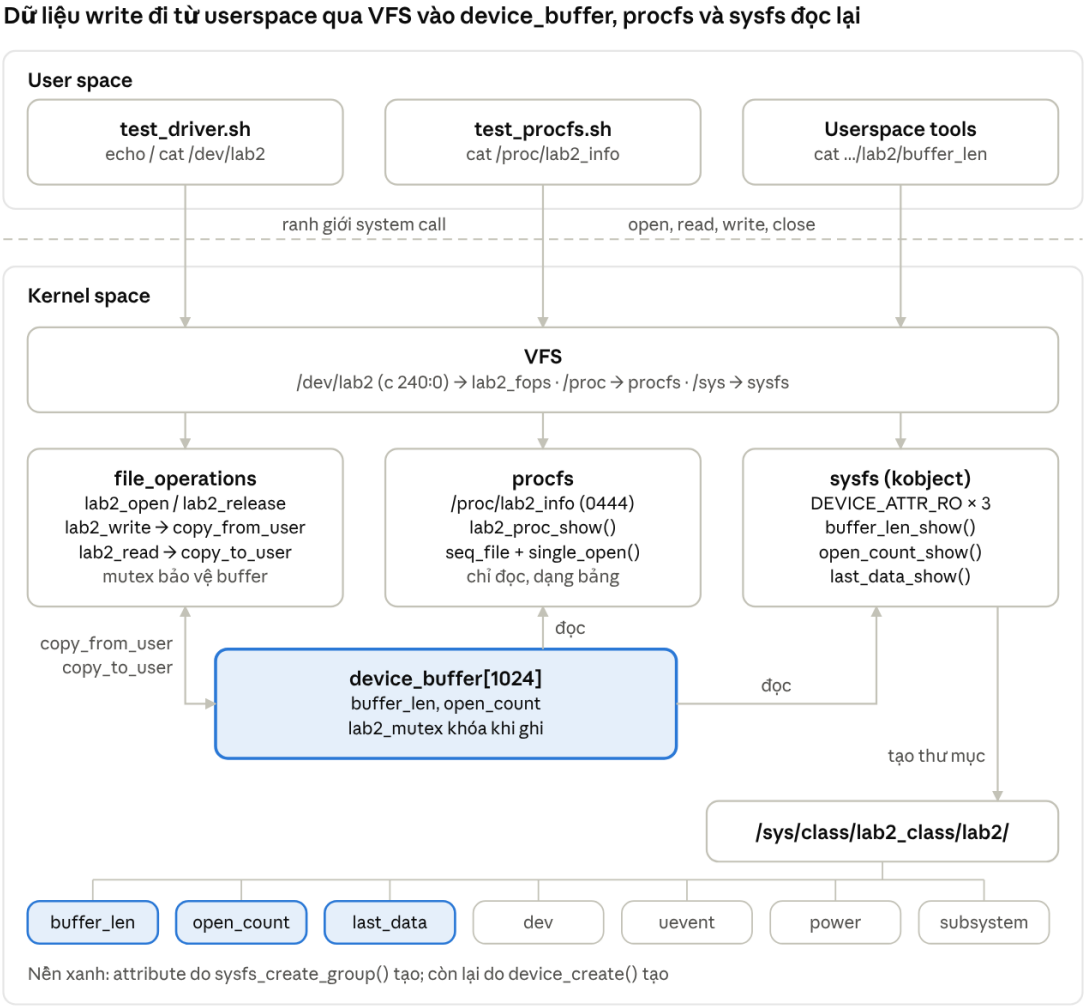
## 1. Thông tin sinh viên và môi trường

Toàn bộ 4 phần A, B, C, D đều chạy thành công: module tự load khi boot QEMU, procfs/sysfs đọc đúng dữ liệu, JFFS2 giữ dữ liệu sau umount/mount.

| Mục | Giá trị |
|---|---|
| Họ và tên | Nguyễn Ngọc Quang |
| MSSV | SE202007 |
| Môn học | Embedded Linux Systems – LAB-02 |
| Ngày thực hiện | 25/09/2026 – 06/10/2026 |

## 2. Sơ đồ kiến trúc driver

Một lệnh `echo ... > /dev/lab2` đi qua system call vào VFS, VFS gọi `lab2_write()` trong `lab2_fops`, hàm này dùng `copy_from_user()` chép dữ liệu vào `device_buffer`. procfs và sysfs không giữ dữ liệu riêng mà đọc lại chính `device_buffer`, `buffer_len`, `open_count` mỗi lần được `cat`.




Kiến trúc lab2_driver · luồng write và cây sysfs

Luồng write: user buffer → `write()` → VFS → `lab2_write()` → `copy_from_user()` → `device_buffer` (giữ `lab2_mutex`) → `/proc/lab2_info` và `/sys/class/lab2_class/lab2/*` hiển thị giá trị mới. Luồng read đi ngược lại qua `copy_to_user()`.

## 3. Log thực hành

Các trích đoạn dưới đây lấy nguyên văn từ terminal. Log đầy đủ nằm trong `logs/boot_log.txt`, `logs/driver_test.txt`, `logs/procfs_test.txt` và `logs/mtd_jffs2.txt`.

### Phần A – Kernel module character device

Module build cho ARM, vermagic khớp kernel target:

```
$ modinfo lab2_driver.ko
version:        1.0
```

```
description:    LAB-02: Simple Character Device Driver
author:         Embedded Linux Lab
license:        GPL
name:           lab2_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8

$ file lab2_driver.ko
lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), with debug_info, not stripped
```

Nạp module và kiểm tra trong QEMU:

```
/ # lsmod
lab2_driver 16384 0 - Live 0x7f000000 (O)
/ # ls -la /dev/lab2
crw-rw-rw-    1 root     0        240,   0 Oct  6 11:19 /dev/lab2
```

Đọc/ghi `/dev/lab2` qua `test_driver.sh` (trích):

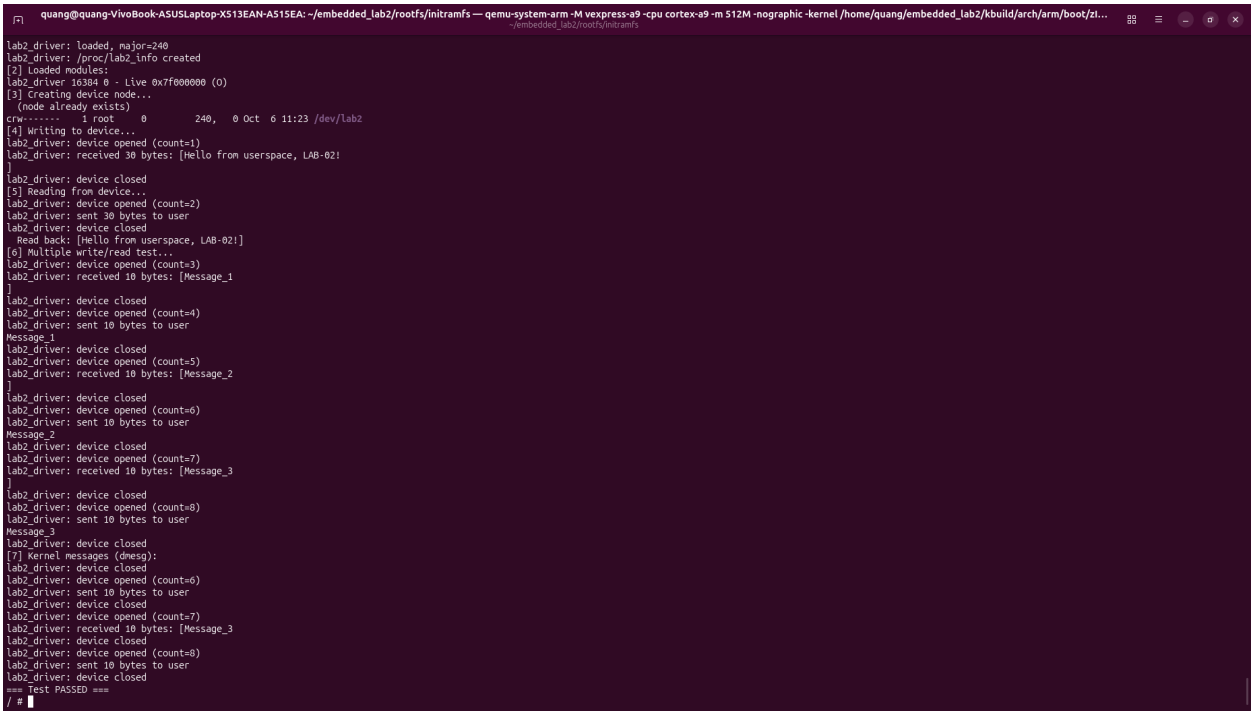
```
[4] Writing to device...
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!]
lab2_driver: device closed
[5] Reading from device...
lab2_driver: sent 30 bytes to user
  Read back: [Hello from userspace, LAB-02!]
[6] Multiple write/read test...
lab2_driver: received 10 bytes: [Message_1]
lab2_driver: sent 10 bytes to user
Message_1
...
Message_3
=== Test PASSED ===
```




*Hình 1. modinfo và file lab2_driver.ko: module ARM EABI5, vermagic 5.15.0 SMP mod_unload ARMv7*



*Hình 2. Module tự load khi boot, lsmod thấy lab2_driver, /dev/lab2 là character device 240:0*




*Hình 3. test_driver.sh: ghi/đọc /dev/lab2 đúng dữ liệu, kết thúc bằng Test PASSED*

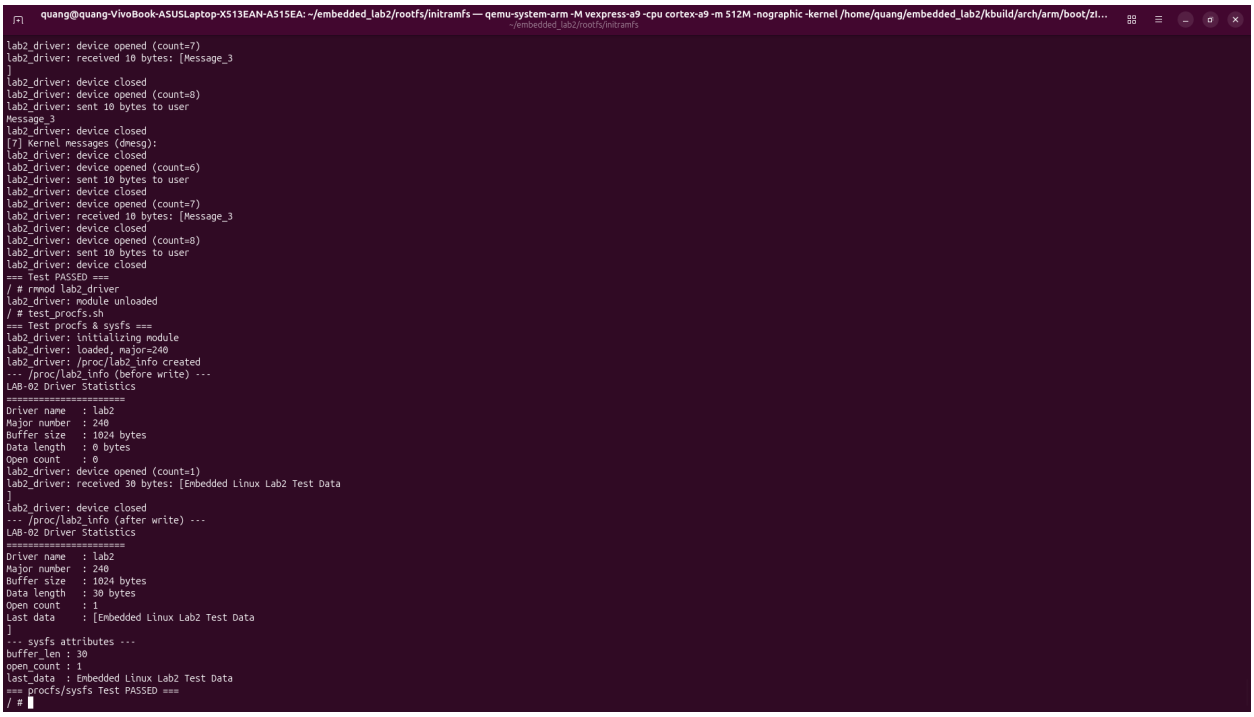
## Phần B – procfs và sysfs

`/proc/lab2_info` trước và sau khi ghi 30 byte vào `/dev/lab2`:

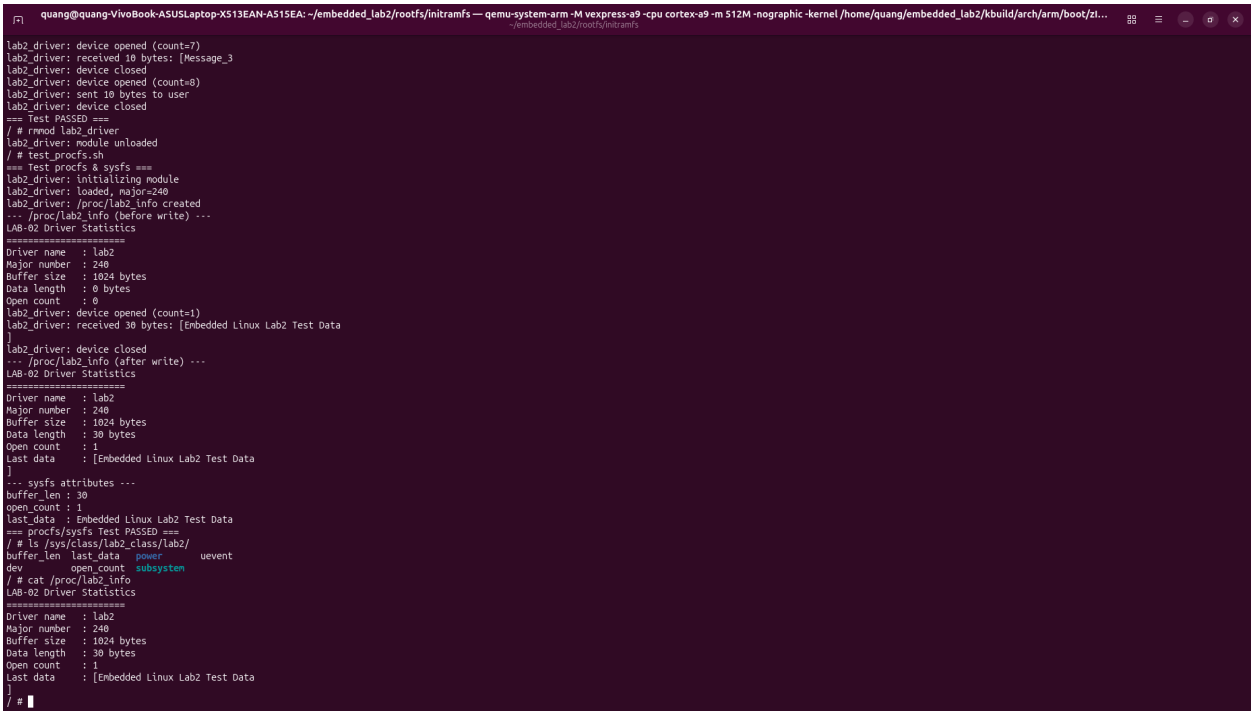
```
--- /proc/lab2_info (before write) ---
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 0 bytes
Open count    : 0
--- /proc/lab2_info (after write) ---
Data length   : 30 bytes
Open count    : 1
Last data     : [Embedded Linux Lab2 Test Data]
```

Các attribute sysfs khớp với procfs:

```
/ # ls /sys/class/lab2_class/lab2/
buffer_len  dev  last_data  open_count  power  subsystem  uevent
buffer_len : 30
open_count : 1
last_data  : Embedded Linux Lab2 Test Data
=== procfs/sysfs Test PASSED ===
```




*Hình 4. test_procfs.sh: /proc/lab2_info trước và sau khi ghi, các thuộc tính sysfs, procfs/sysfs Test PASSED*




*Hình 5. ls /sys/class/lab2_class/lab2/ và cat /proc/lab2_info*

**Phần C – MTD và JFFS2 (chạy trên host Ubuntu bằng sudo, không phải trong QEMU)**

Máy host đã có sẵn `mtd0` là BIOS thật, nên nandsim xuất hiện là `mtd1`. Mọi thao tác chỉ thực hiện trên `mtd1`/`mtdblock1`.

```
$ cat /proc/mtd
mtd0: 01000000 00001000 "BIOS"
mtd1: 02000000 00004000 "NAND simulator partition 0"

$ sudo mtdinfo /dev/mtd1
Type:                   nand
Eraseblock size:            16384 bytes, 16.0 KiB
Amount of eraseblocks:      2048 (33554432 bytes, 32.0 MiB)
Minimum input/output unit size: 512 bytes
OOB size:                   16 bytes

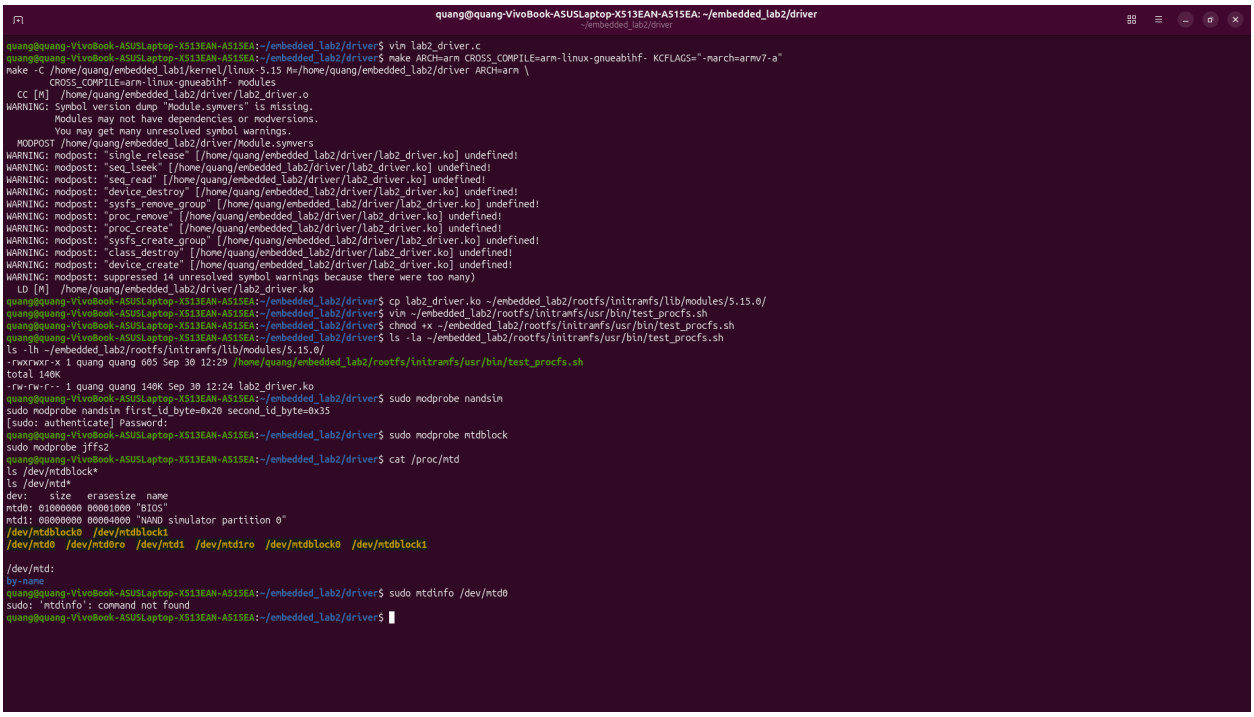
$ sudo flash_erase /dev/mtd1 0 0
Erasing 32768 Kibyte @ 0 -- 100 % complete
$ sudo nandwrite -p /dev/mtd1 /tmp/rootfs.jffs2
Writing data to block 0 at offset 0x0
```

Mount, ghi thêm dữ liệu, rồi umount/mount lại để kiểm tra persistence:

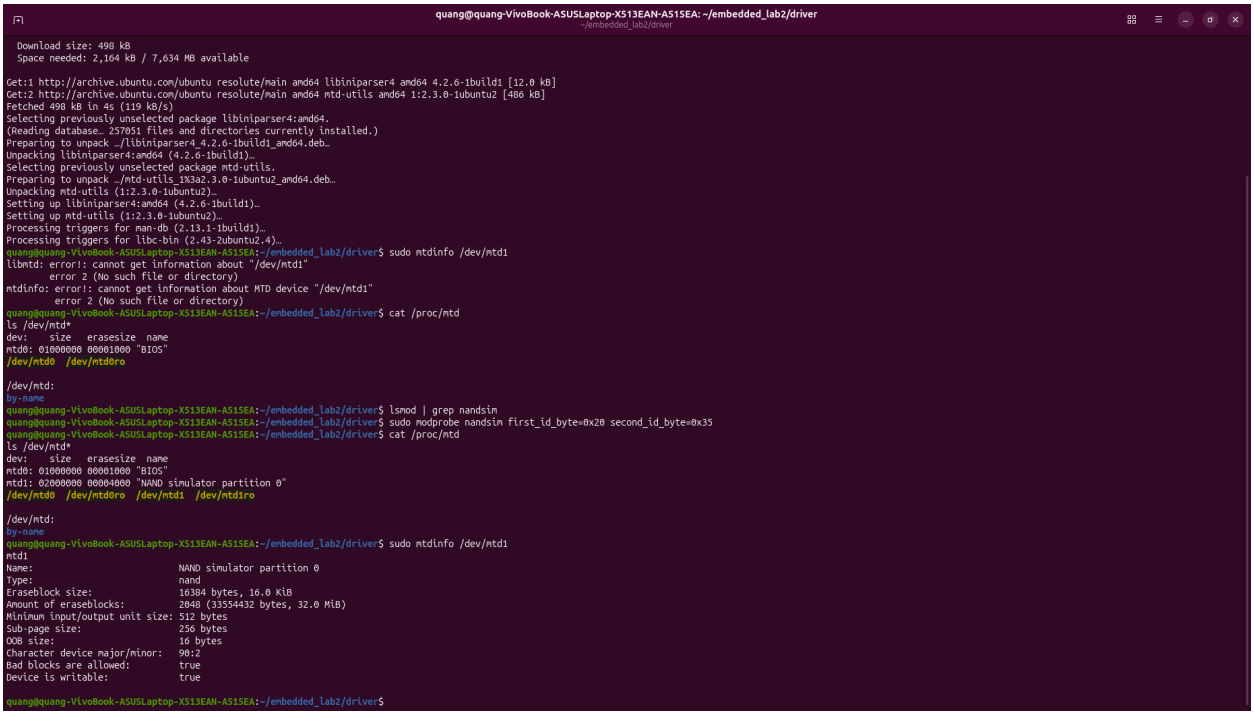
```
$ cat /mnt/jffs2/hello.txt /mnt/jffs2/info.txt
Hello from JFFS2!
Embedded Linux Lab2
# sau khi umount và mount lại /dev/mtdblock1:
$ cat /mnt/jffs2/logs/boot.log /mnt/jffs2/runtime.txt
Tue Oct  6 06:33:09 PM +07 2026
Written at Tue Oct  6 06:33:09 PM +07 2026
New runtime file
```

Dump raw flash cho thấy magic JFFS2 `0x1985` (`85 19`, little-endian) ngay ở byte đầu:

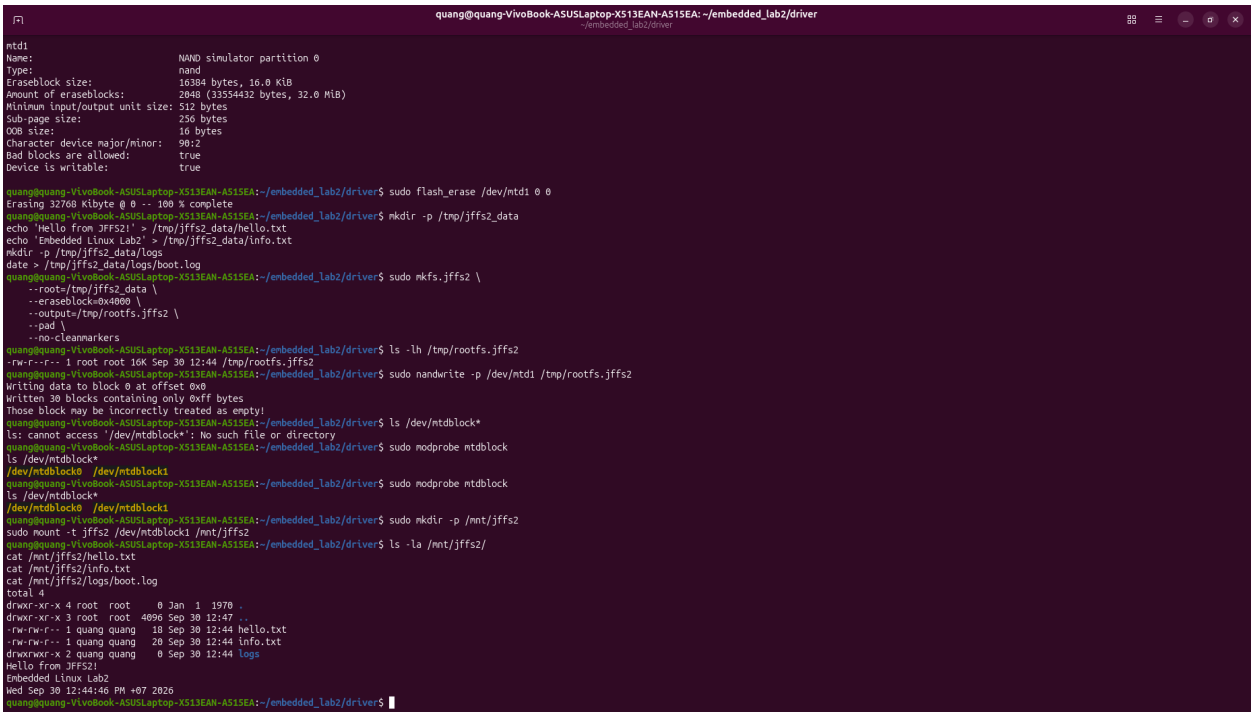
```
00000000  85 19 01 e0 31 00 00 00  1d d9 82 42 01 00 00 00  |....1.....B....|
00000020  60 7e 6f 2b 15 da 69 f4  68 65 6c 6c 6f 2e 74 78  |`~o+..i.hello.tx|
00000070  ee dd 4b b3 8f 1d 5a 6f  48 65 6c 6c 6f 20 66 72  |..K...ZoHello fr|
```



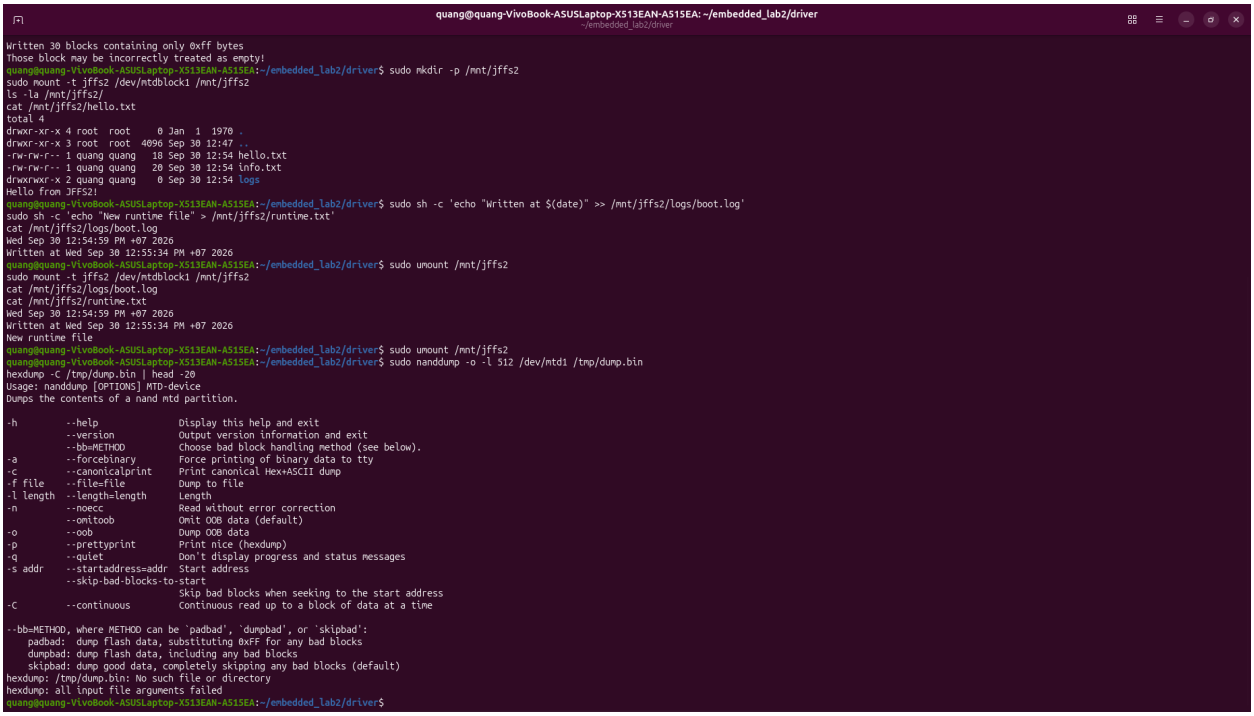

*Hình 6. cat /proc/mtd: mtd0 là BIOS thật của máy, nandsim xuất hiện là mtd1*



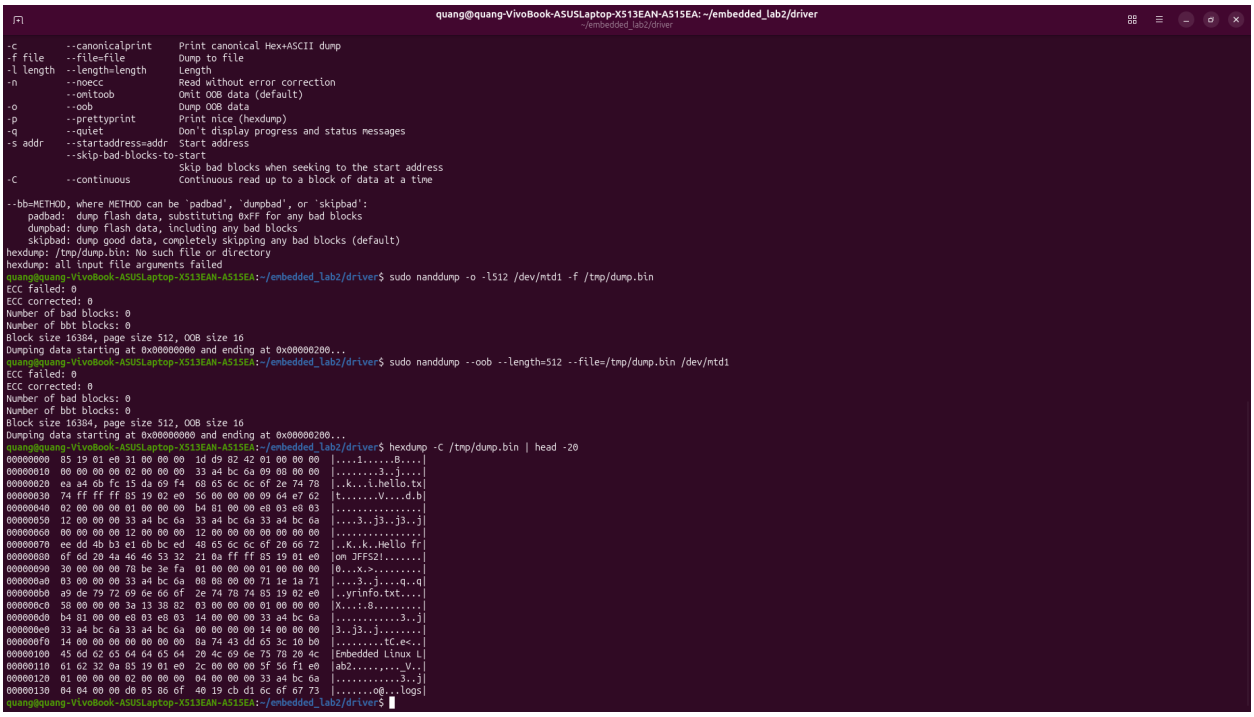

*Hình 7. mtdinfo /dev/mtd1: NAND 32 MiB, eraseblock 16 KiB, 2048 block*




*Hình 8. Mount JFFS2 từ /dev/mtdblock1 và đọc lại dữ liệu đã ghi*




*Hình 9. Persistence: ghi thêm dữ liệu, umount rồi mount lại, dữ liệu vẫn còn*



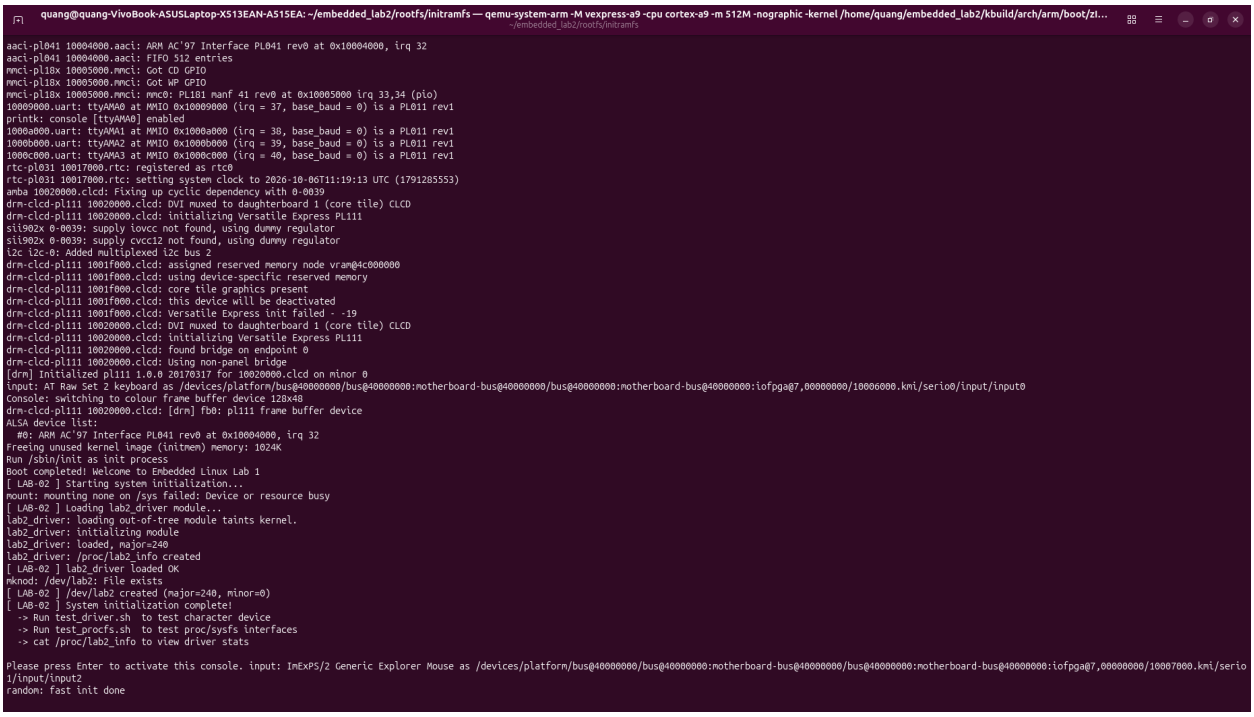

*Hình 10. nanddump + hexdump: magic JFFS2 85 19 ở byte đầu tiên*

## Phần D – BusyBox init tự load module

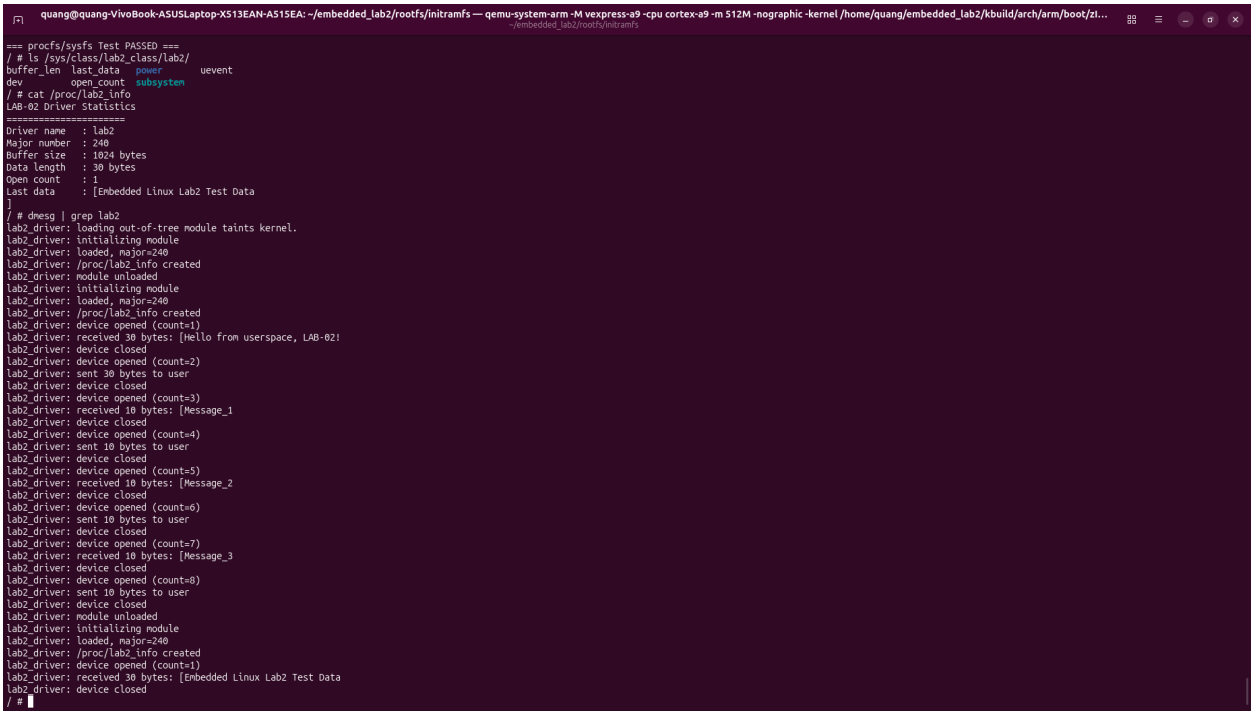
Boot log QEMU cho thấy `rcS` tự nạp module và tạo device node:

```
Run /sbin/init as init process
[ LAB-02 ] Starting system initialization...
[ LAB-02 ] Loading lab2_driver module...
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: /proc/lab2_info created
[ LAB-02 ] lab2_driver loaded OK
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)
[ LAB-02 ] System initialization complete!
```

`dmesg | grep lab2` ghi lại trọn vòng đời: load, 8 lần open/read/write/close của `test_driver.sh`, module unloaded khi `rmmod`, rồi load lại cho `test_procfs.sh`.




*Hình 11. Boot log QEMU: rcS tự nạp lab2_driver và tạo /dev/lab2*




*Hình 12. dmesg | grep lab2: vòng đời module (load, open/read/write/close, unload)*

## 4. So sánh filesystem

Với thiết bị IoT 32 MB raw NAND cần OTA, khuyến nghị là **UBI + SquashFS (rootfs chỉ đọc, 2 slot A/B) + UBIFS (dữ liệu ghi được)**.

| Tiêu chí | ext4 | JFFS2 | cramfs | SquashFS | UBIFS (trên UBI) |
|---|---|---|---|---|---|
| Ghi được (writable) | Có | Có | Không | Không | Có |
| Thiết bị phù hợp | eMMC, SD, SSD (có FTL) | Raw NAND/NOR nhỏ | ROM, NOR nhỏ | Mọi block device, UBI volume | Raw NAND vừa và lớn |
| Wear leveling | Không tự làm, dựa vào FTL | Có, trong filesystem | Không cần (chỉ đọc) | Không cần (chỉ đọc) | Có, ở tầng UBI, trên toàn chip |
| Quản lý bad block | Không | Có | Không | Không | Có (UBI) |
| Nén | Không | Có (zlib, lzo) | Có (zlib) | Có (gzip, xz, lzo, zstd) | Có (lzo, zlib, zstd) |
| Thời gian mount/boot | Nhanh | Chậm, quét toàn bộ flash, tăng theo dung lượng | Nhanh | Nhanh | Nhanh, có index trên flash |
| Chịu mất điện | Journal | Log-structured, tốt | Không ghi nên không hỏng | Không ghi nên không hỏng | Tốt, có journal |
| Trong lab này | Không dùng | Đã format, mount, kiểm tra persistence trên nandsim 32 MB | Không dùng | Không dùng | Không dùng |

**Khuyến nghị cho IoT 32 MB NAND + OTA:**

- Không dùng ext4: raw NAND không có FTL, ext4 sẽ ghi đè tại chỗ, không wear leveling và không xử lý bad block.
- Dùng UBI làm lớp quản lý flash: wear leveling trên toàn chip, xử lý bad block, chia volume linh hoạt.
- Rootfs đặt trên SquashFS (qua `ubiblock`), chia 2 volume A/B. OTA ghi bản mới vào slot không chạy bằng `ubiupdatevol`, đổi biến boot của U-Boot, lỗi thì quay lại slot cũ. SquashFS nén tốt (xz), giúp 2 bản rootfs vừa 32 MB.
- Dữ liệu cấu hình và log đặt trên một volume UBIFS nhỏ, ghi được và chịu mất điện.

- JFFS2 (đã dùng trong lab) vẫn chạy được ở 32 MB, nhưng mount phải quét toàn flash và không có cơ chế cập nhật nguyên khối volume, nên kém hơn cho OTA. cramfs đã lỗi thời so với SquashFS.

## 5. Trả lời câu hỏi phân tích

### Câu 1. Vì sao không dùng memcpy() giữa kernel buffer và user buffer?

Kernel và tiến trình dùng hai không gian địa chỉ ảo khác nhau. Kernel trong lab chia 2G/2G: user space nằm ở `0x00000000–0x7EFFFFFF`, module ở ngay dưới `0x80000000` (`lab2_driver` nạp tại `0x7f000000`), còn kernel từ `0x80000000` trở lên (backtrace có địa chỉ `[<8010f670>]`).

Con trỏ `buf` trong `lab2_read()`/`lab2_write()` là địa chỉ do tiến trình truyền vào, kernel không thể tin nó:

- Trang nhớ có thể chưa được map hoặc đã bị swap, truy cập sẽ gây page fault.
- Tiến trình có thể cố ý truyền địa chỉ trỏ vào vùng kernel để đọc/ghi trái phép.

`copy_from_user()`/`copy_to_user()` kiểm tra địa chỉ có thuộc user space không (`access_ok`), bật quyền truy cập user từ kernel (trên ARM là cơ chế domain/PAN), và dùng bảng exception fixup: nếu page fault xảy ra, hàm trả về số byte chưa copy thay vì làm kernel crash. Driver dựa vào đó để trả `-EFAULT`, như trong code `if (copy_to_user(...)) return -EFAULT;`.

Nếu dùng `memcpy()`: con trỏ hợp lệ thì có thể chạy được, nhưng con trỏ sai sẽ gây Oops hoặc kernel panic, và mở lỗ hổng cho tiến trình đọc/ghi bộ nhớ kernel. Đây cũng là lý do `test_driver.sh` ghi rồi đọc lại đúng 30 byte `Hello from userspace, LAB-02!` an toàn qua hai hàm này.

### Câu 2. Vì sao cần mutex, và race condition khi hai process cùng write()?

`device_buffer`, `buffer_len` và `open_count` là biến toàn cục, mọi process mở `/dev/lab2` đều dùng chung. Kernel có thể chạy hai lời gọi `write()` song song (QEMU chạy `-smp 2`) hoặc chuyển tiến trình giữa chừng, vì `copy_from_user()` có thể ngủ khi gặp page fault.

Race condition cụ thể khi không có mutex:

1. Process A ghi `"AAAAAAAAAA"` (10 byte), copy xong vào `device_buffer` nhưng chưa kịp gán `buffer_len = 10`.
2. Process B ghi `"BBB"` (3 byte), copy đè 3 byte đầu và gán `buffer_len = 3`.
3. A tiếp tục, gán `buffer_len = 10`.

Kết quả: buffer chứa `"BBBAAAAAAA"` với độ dài 10, dữ liệu trộn lẫn của hai process. `/proc/lab2_info` và sysfs `last_data` sẽ báo sai. Tương tự, `open_count++` không nguyên tử có thể mất một lần đếm. Trong lab, `open_count` tăng đúng từ 1 đến 8 qua 8 lần mở liên tiếp vì có `mutex_lock()` quanh thao tác tăng biến.

Dùng mutex (thay vì spinlock) vì đoạn găng có `copy_from_user()`, hàm có thể ngủ. Lưu ý code mẫu của lab chưa khóa trong `lab2_read()`, nên một lần đọc song song với ghi vẫn có thể đọc dữ liệu đang ghi dở.

### Câu 3. So sánh procfs và sysfs

| Tiêu chí | procfs (/proc) | sysfs (/sys) |
|---|---|---|
| Mục đích ban đầu | Thông tin tiến trình | Mô hình thiết bị (device model) |
| Cấu trúc | Phẳng, mỗi driver tự đặt tên | Cây theo bus/class/device, gắn với kobject |
| Nội dung một file | Nhiều dòng, định dạng tự do | Một giá trị một file |
| Trong lab | /proc/lab2_info: 6 dòng thống kê | buffer_len, open_count, last_data |

Kernel documentation khuyến nghị không thêm file mới vào /proc vì trước đây /proc bị lạm dụng thành nơi chứa mọi thứ, định dạng tự do khó parse và khó giữ ổn định ABI. sysfs có quy tắc rõ: một giá trị một file, nằm đúng chỗ trong cây thiết bị, tự xuất hiện và biến mất cùng device. Trong lab, thư mục /sys/class/lab2_class/lab2/ còn có sẵn dev, uevent, subsystem do kernel tạo khi gọi device_create().

Phân chia hợp lý cho driver này:

- /proc: bảng tóm tắt cho người đọc, như /proc/lab2_info (tên driver, major, kích thước buffer). Dùng để debug nhanh bằng cat.
- /sys: từng thuộc tính máy đọc được, như buffer_len, open_count, last_data. Script hoặc chương trình userspace chỉ cần đọc một số, không cần parse.

Kết quả lab cho thấy hai giao diện nhất quán: sau khi ghi, /proc báo Data length: 30 bytes, sysfs báo buffer_len: 30.

### Câu 4. Vì sao NAND phải erase trước khi ghi, và JFFS2 xử lý thế nào?

NAND lưu bit bằng điện tích trong floating gate. Thao tác program chỉ có thể đổi bit từ 1 sang 0. Muốn đưa bit về 1 phải erase, và erase chỉ làm được trên cả một erase block. Với nandsim trong lab, page là 512 byte nhưng erase block là 16 KiB (Eraseblock size: 16384 bytes). Sau flash_erase, toàn bộ flash là 0xFF. Mỗi block chỉ chịu được số lần erase giới hạn (khoảng $10^4$–$10^5$ chu kỳ), và NAND có thể có bad block ngay từ khi xuất xưởng.

ext4 ghi trực tiếp được vì nó chạy trên block device có FTL (eMMC, SD, SSD). FTL ẩn việc erase, làm wear leveling và ánh xạ sector logic sang trang vật lý. Raw NAND (/dev/mtd1) không có lớp đó.

JFFS2 là log-structured filesystem:

- Không ghi đè. Mỗi thay đổi được ghi thành node mới, có version cao hơn, vào chỗ trống tiếp theo trong log.
- Node cũ trở thành rác (obsolete). Garbage collector chọn block nhiều rác, chép node còn dùng sang chỗ khác, rồi erase cả block.
- Việc ghi rải đều trên các block nên có wear leveling, và block hỏng được đánh dấu bỏ qua.
- Khi mount, JFFS2 quét toàn bộ flash để dựng lại cây file từ các node có version mới nhất.

Lab kiểm chứng điều này: thêm dòng `Written at ...` và file `runtime.txt`, umount rồi mount lại, dữ liệu vẫn còn nguyên. `hexdump` thấy magic `85 19` và các node chứa `hello.txt`, `Hello from JFFS2!` nằm liên tiếp trong log.

### Câu 5. insmod lỗi trong rcS thì sao, và cải thiện rcS

BusyBox init chạy `::sysinit:/etc/init.d/rcS`, đợi script kết thúc, rồi chạy tiếp các mục `askfirst`. Init **không quan tâm mã thoát** của rcS. Vì vậy `insmod` lỗi không làm dừng boot: hệ thống vẫn vào shell. Lab có ví dụ thực tế: dòng `mount: mounting none on /sys failed` và `mknod: /dev/lab2: File exists` đều báo lỗi nhưng boot vẫn tiếp tục tới `/ #`.

Để phân biệt lỗi, rcS cần tự kiểm tra:

- **Lỗi nghiêm trọng** (không mount được `/proc`, thiếu `/dev/console`): hệ thống không dùng được, nên in lỗi rõ và mở shell cứu hộ thay vì chạy tiếp.
- **Lỗi không nghiêm trọng** (module phụ không load): ghi cảnh báo, lưu vào log, bỏ qua phần phụ thuộc rồi boot tiếp.

rcS cải tiến:

```
#!/bin/sh
LOG=/tmp/boot.log
MOD=/lib/modules/$(uname -r)/lab2_driver.ko

fatal() { echo "[ FATAL ] $1"; echo "Dropping to emergency shell"; exec /bin/sh; }
warn()  { echo "[ WARN ] $1" | tee -a $LOG; }

# Lỗi nghiêm trọng: thiếu proc thì dừng
mountpoint -q /proc || mount -t proc none /proc || fatal "cannot mount /proc"
mountpoint -q /sys  || mount -t sysfs none /sys || warn "cannot mount /sys"
mount -t tmpfs none /tmp
mount -t devtmpfs none /dev 2>/dev/null
[ -c /dev/console ] || mknod /dev/console c 5 1 || fatal "no /dev/console"

# Lỗi không nghiêm trọng: module phụ
if [ ! -f "$MOD" ]; then
   warn "lab2_driver.ko not found, skipping"
elif grep -q '^lab2_driver ' /proc/modules; then
   echo "[ LAB-02 ] lab2_driver already loaded"
elif ! insmod "$MOD" 2>>$LOG; then
   warn "insmod lab2_driver failed (see $LOG, dmesg)"
else
   [ -c /dev/lab2 ] || mknod /dev/lab2 c 240 0
   chmod 666 /dev/lab2
   echo "[ LAB-02 ] lab2_driver loaded OK"
fi

echo "[ LAB-02 ] System initialization complete"
```

Các cải tiến: dùng `uname -r` thay vì cố định `5.15.0`, kiểm tra module đã load để tránh `File exists`, chỉ tạo `/dev/lab2` khi chưa có, ghi lỗi vào `/tmp/boot.log`, và tách rõ đường lỗi nghiêm trọng (`fatal`) với cảnh báo (`warn`).

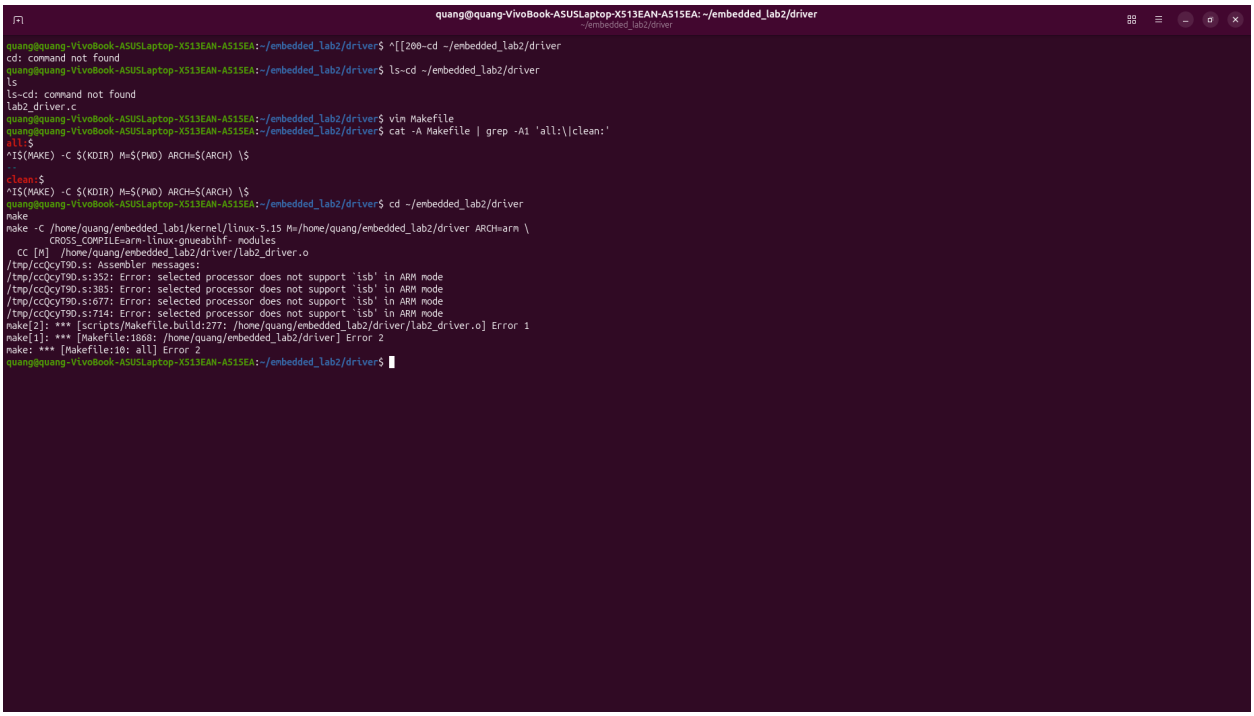
## 6. Nhận xét và khó khăn

Khó khăn lớn nhất là kernel LAB-01 build bằng GCC 15 không boot được, phải build lại bằng GCC 12 với soft-float. Các lỗi gặp phải và cách xử lý:

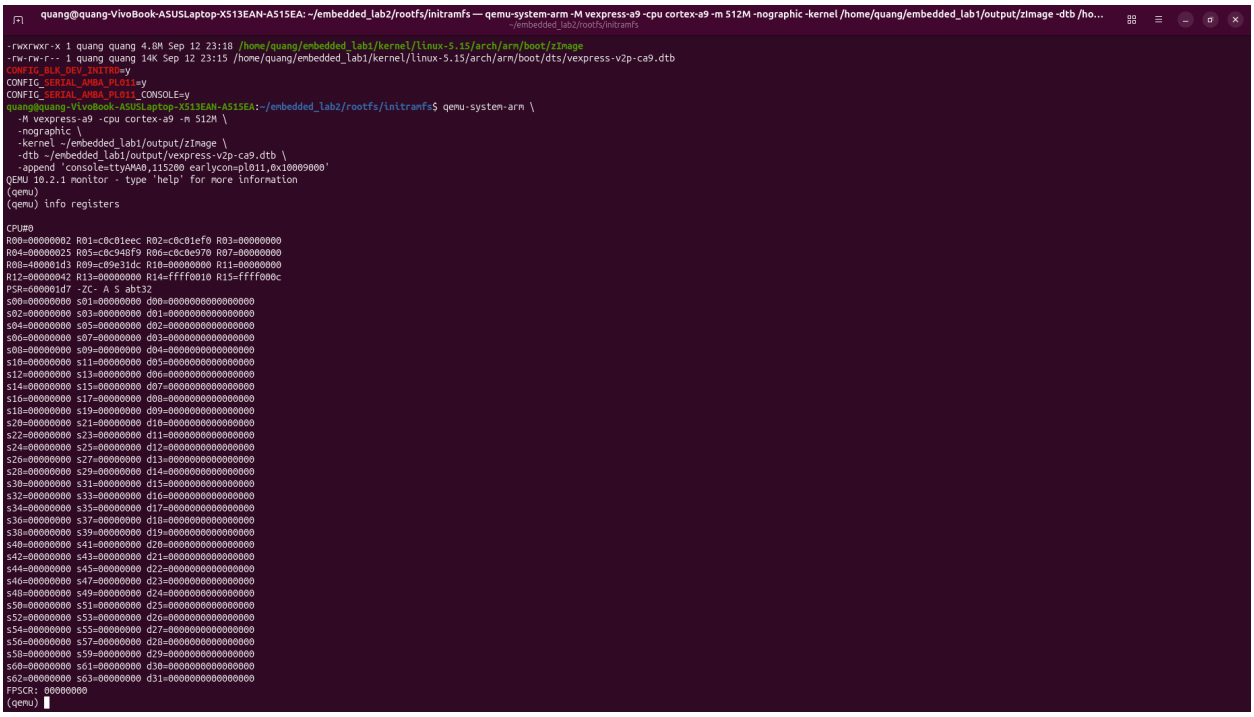
| # | Hiện tượng | Nguyên nhân | Cách giải quyết |
|---|---|---|---|
| 1 | Build module báo `selected processor does not support 'isb' in ARM mode` | GCC 15 mặc định `armv7-a+fp` + `-mfloat-abi=hard`. Kernel thử `-march=armv7-a` (không FPU) bị GCC từ chối (`selected architecture lacks an FPU`), nên rơi về `armv5t` | Tạm thời thêm `KCFLAGS="-march=armv7-a"`. Cách triệt để: dùng `arm-linux-gnueabihf-gcc-12 -mfloat-abi=soft` |
| 2 | QEMU chạy (`VM status: running`) nhưng không in gì | Kernel LAB-01 build sai kiến trúc vì lỗi số 1, crash ngay khi khởi động. `info registers` cho thấy PC = `0xffff000c`, chế độ `abt32` (Prefetch Abort) | Build lại kernel vào thư mục riêng: `make O=~/embedded_lab2/k build ... CC="arm-linux-gnueabihf-gcc-12 -mfloat-abi=soft"`. Kernel mới boot được ngay |
| 3 | Build module báo `No rule to make target 'scripts/module.lds'` | Source kernel chưa có các file phụ trợ cho build module ngoài | Chạy `make modules_prepare` trong source kernel |
| 4 | Boot initramfs panic `VFS: Unable to mount root fs on unknown-block(0,0)` | Rootfs thiếu symlink BusyBox: `sbin/` trống, không có `sbin/init` và `bin/sh`. `busybox --install` không chạy được trên host x86 | Tạo symlink thủ công `ln -sf ../bin/busybox sbin/init`, `ln -sf busybox bin/sh` và các applet cần dùng, rồi đóng gói lại |
| 5 | `cat /proc/mtd` thấy `mtd0` là `"BIOS"` | Máy host có sẵn flash BIOS dạng MTD, nandsim trở thành `mtd1` | Chỉ thao tác trên `mtd1/mtdblock1`. Script Phần C tự tìm số MTD theo tên `NAND simulator` |

| # | Hiện tượng | Nguyên nhân | Cách giải quyết |
|---|---|---|---|
| 6 | cp rootfs báo Operation not permitted | dev/console, dev/null, dev/zero là device node, cần quyền root | sudo cp -a rồi sudo chown -R |
| 7 | Máy tắt đột ngột giữa chừng | Mất nandsim và mount JFFS2 (chỉ nằm trong RAM), vim để lại swap file | Khôi phục bằng vim R (Recover). Làm lại Phần C liền mạch bằng một script, ghi log bằng tee |
| 8 | test_driver.sh báo insmod: File exists | rcS đã tự load module khi boot | rmmod lab2_driver trước khi chạy script. Câu 5 đề xuất rcS kiểm tra module đã load chưa |

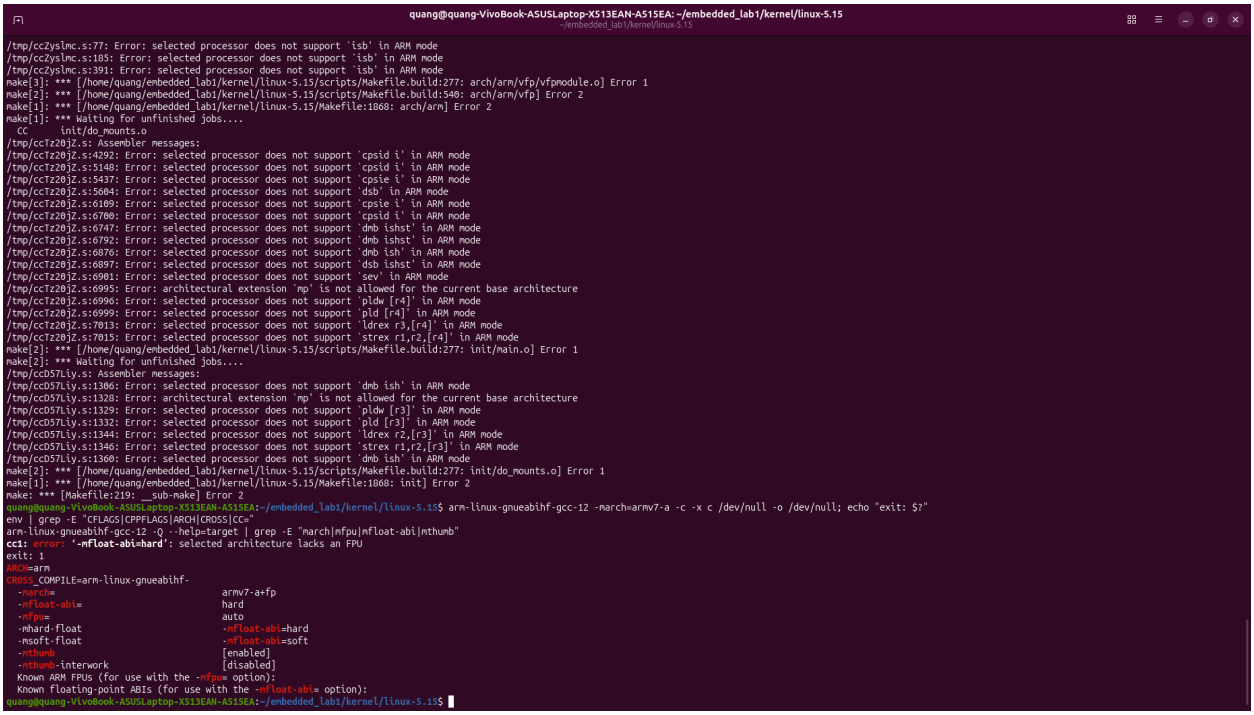
**Ảnh minh họa các lỗi chính:**



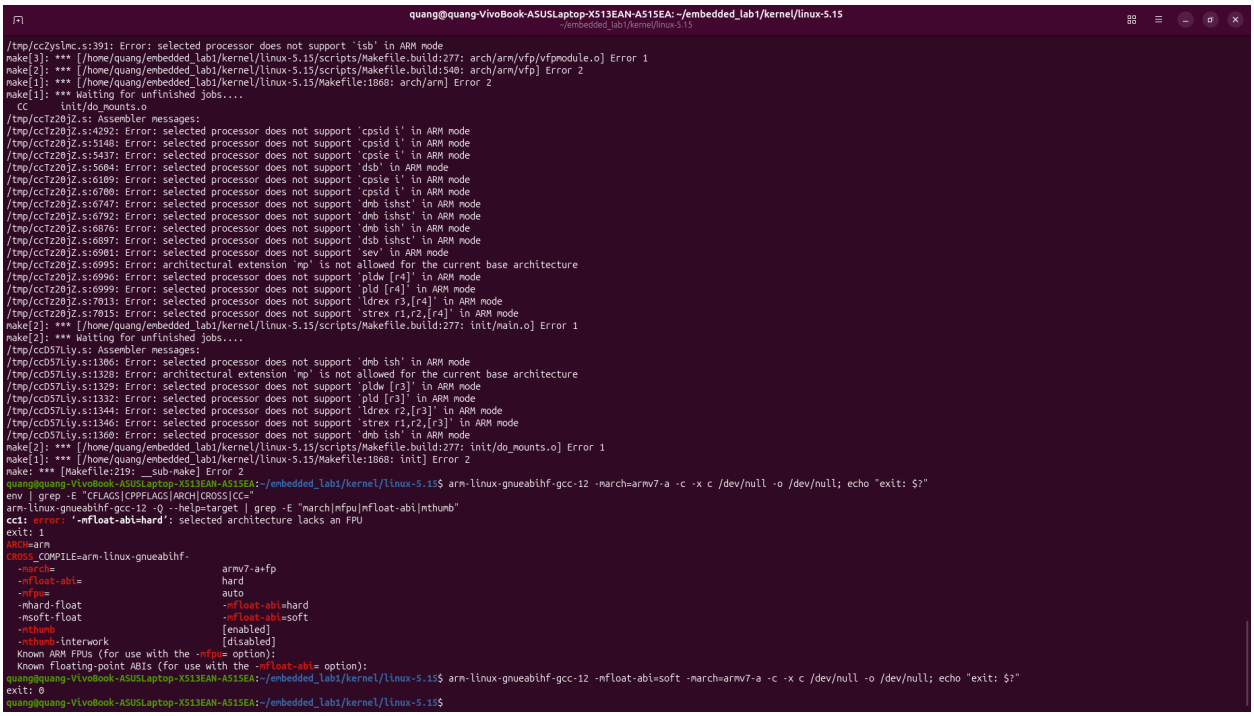

*Hình 13. Lỗi 1: build module báo selected processor does not support isb in ARM mode*



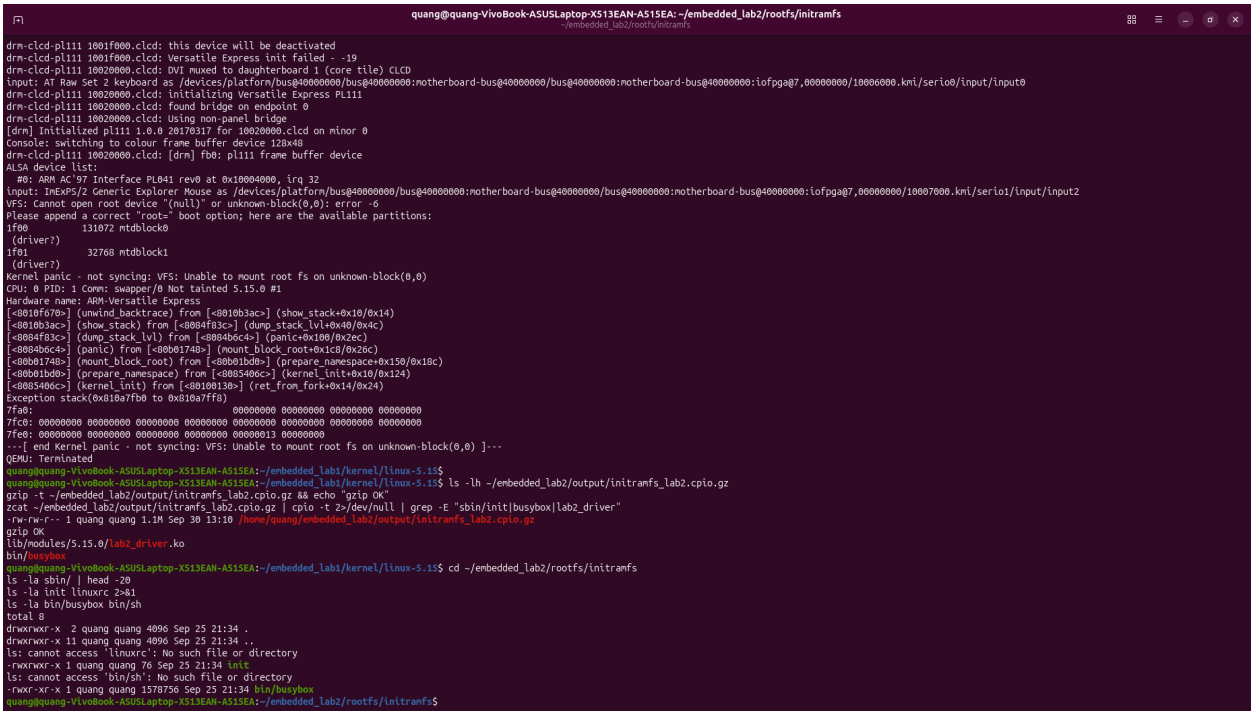

*Hình 14. Lỗi 2: QEMU không in gì; info registers cho thấy PC = 0xffff000c, chế độ abt32 (kernel crash sớm)*




*Hình 15. Nguyên nhân lỗi 1–2: -march=armv7-a bị GCC từ chối với hard-float (exit 1)*




*Hình 16. Cách sửa: thêm -mfloat-abi=soft thì kiểm tra thành công (exit 0)*




*Hình 17. Lỗi 4: kernel panic do sbin/ trong rootfs trống, thiếu sbin/init*

**Nhận xét:**

- Kiểm tra toolchain trước khi build: một cờ ABI sai không báo lỗi ở bước build kernel, nhưng làm kernel crash âm thầm khi chạy.

- Khi QEMU im lặng, QEMU monitor (`Ctrl+A C`, `info registers`) giúp xác định kernel đang crash hay chỉ không có console.
- Code mẫu còn hai điểm có thể cải thiện: `lab2_read()` không khóa mutex, và `rcS` dùng đường dẫn `5.15.0` cố định.